

Tema: Vamos de Paseo

Reporte de Contribuciones - Integrante 2 (Dijkstra Ruta Mas Corta)

Asignatura:	Estructuras de Datos y Algoritmos
Estudiante (Integrante 2):	Responsable de Algoritmos Dijkstra
Algoritmos implementados:	Dijkstra para Ruta Mas Corta (entre 2 ciudades y de origen a todos)
Lenguaje / Estructura:	Java (Lista de Adyacencia, PriorityQueue)

1. Rol y Responsabilidades del Integrante 2

El **Integrante 2** es el encargado de desarrollar la logica central para determinar la **ruta mas corta** en distancia (kilometros) entre las localidades representadas en el grafo. Esto incluye:

- Diseñar y programar el algoritmo de Dijkstra clasico, el cual minimiza la suma acumulada de los pesos (distancia en km) de las aristas (tramos) que conectan las ciudades.
- Implementar el metodo **rutaMasCorta(String origen, String destino)** para calcular el camino optimo entre dos puntos especificos, reconstruyendo la secuencia de ciudades y mostrando la distancia total.
- Implementar el metodo **rutaMasCortaDesdeOrigen(String origen)** para calcular las distancias minimas y caminos desde una ciudad de partida hacia todas las demas localidades alcanzables en el grafo.
- Estructurar e integrar la Cola de Prioridad (PriorityQueue) utilizando una clase auxiliar comparable para garantizar una complejidad algoritmica optima de $O((V + E) \log V)$.

2. Explicacion Tecnica del Algoritmo de Dijkstra

El algoritmo de Dijkstra resuelve el problema del camino mas corto de fuente unica en un grafo ponderado no dirigido sin pesos negativos. Funciona de manera codiciosa (greedy) seleccionando iterativamente el vertice con la menor distancia conocida provisional y explorando sus conexiones.

Funcionamiento Paso a Paso:

1. **Inicializacion:** Se crea un mapa de *distancias* inicializando todos los vertices con valor 'infinito' (Double.MAX_VALUE), a excepcion de la ciudad origen que se establece en 0. Se inicializa una cola de prioridad con el nodo origen.
2. **Exploracion:** Mientras la cola de prioridad no este vacia, se extrae el nodo con la distancia acumulada mas corta. Si el nodo ya fue visitado con un camino menor, se descarta.

3. **Relajacion:** Para cada tramo vecino que sale del nodo actual, se calcula una nueva distancia tentativa: $distancia_actual + distancia_tramo$. Si este resultado es menor que la distancia guardada previamente para el vecino, se actualiza el mapa de distancias, se guarda al nodo actual como su predecesor ('padre') y se añade a la cola de prioridad.
4. **Reconstruccion:** Si se busca un destino especifico, al llegar a el se retrocede a traves del mapa de predecesores desde el destino hasta el origen, invirtiendo la lista para obtener el camino correcto.

3. Estructura del Codigo Java Implementado

Para dar soporte a la logica de Dijkstra, se definieron tres estructuras base de forma modular y limpia:

Ciudad.java (Vertice): Representa cada nodo. Almacena el nombre de la ciudad y redefine equals() y hashCode() para asegurar que sea usada correctamente como clave en estructuras de HashMap.

Tramo.java (Arco): Representa las conexiones. Guarda la ciudad destino, la distancia (km) y la cantidad de carriles para permitir la busqueda de caminos de flujo del Integrante 3.

Grafo.java (Estructura y Algoritmos): Mantiene un mapa de ciudades por nombre y la lista de adyacencia (Map<Ciudad, List<Tramo>>). Contiene los métodos de insercion e implementa Dijkstra.

Clase Interna de Prioridad:

Se implemento una clase interna estatica llamada NodoDijkstra para empaquetar una Ciudad junto a su distanciaAcumulada, implementando la interfaz Comparable para que la PriorityQueue de Java los ordene de menor a mayor distancia:

```
private static class NodoDijkstra implements Comparable {
    Ciudad ciudad;
    double distanciaAcumulada;

    public NodoDijkstra(Ciudad ciudad, double distanciaAcumulada) {
        this.ciudad = ciudad;
        this.distanciaAcumulada = distanciaAcumulada;
    }

    @Override
    public int compareTo(NodoDijkstra otro) {
        return Double.compare(this.distanciaAcumulada, otro.distanciaAcumulada);
    }
}
```

4. Resultados y Validacion de Ejecucion

Para comprobar que la logica de Dijkstra funciona correctamente, se creo un escenario en Main.java cargando un grafo de 6 ciudades de Argentina (Buenos Aires, Rosario, Cordoba, Mendoza, San Luis y Santa Fe) conectadas por tramos con diferentes distancias en kilometros y carriles.

Resultados de las Consultas:

Origen	Destino	Camino Encontrado (Dijkstra)	Distancia Total
Buenos Aires	Mendoza	Buenos Aires -> San Luis -> Mendoza	1060.0 km
Santa Fe	San Luis	Santa Fe -> Cordoba -> San Luis	750.0 km
Buenos Aires	Rosario	Buenos Aires -> Rosario	300.0 km
Buenos Aires	Cordoba	Buenos Aires -> Rosario -> Cordoba	700.0 km

Analisis del Caso BA-Mendoza: A pesar de que la ruta *Buenos Aires -> Rosario -> Cordoba -> Mendoza* tiene tramos de mayor capacidad de flujo (carriles), la ruta mas corta elegida por el algoritmo es por *San Luis* (1060 km vs 1300 km), demostrando que Dijkstra minimiza la suma acumulada de distancias correctamente.

5. Propuesta de Guion para la Defensa en Video

Dado que el video explicativo grupal dura alrededor de 5 minutos, cada integrante dispone de aproximadamente 1.5 minutos para exponer. Aquí tienes un guion sugerido para tu parte:

Guion sugerido (Tiempo estimado: 1:30 min):

"Hola a todos. Mi nombre es [Tu Nombre] y soy el Integrante 2. Mi responsabilidad en el proyecto fue implementar el algoritmo de Dijkstra para resolver el problema de la ruta mas corta en distancia. El algoritmo de Dijkstra funciona explorando de manera voraz o greedy el vertice con la menor distancia conocida. Para implementar esto de forma eficiente en Java, cree una clase auxiliar interna llamada NodoDijkstra que asocia una ciudad con su distancia acumulada, implementando la interfaz Comparable. Esto permite que la PriorityQueue organice los nodos de menor a mayor automaticamente. En cada iteracion, extraemos el vertice mas cercano, revisamos sus tramos vecinos y relajamos las distancias en el mapa si encontramos un camino mas corto. Al final, mediante un mapa de predecesores o 'padres', reconstruimos la ruta optima desde el destino hacia atras. Tambien implemente una variante que nos permite calcular las distancias minimas desde una ciudad origen hacia todas las demas. En las pruebas de consola que podemos observar, al buscar la ruta de Buenos Aires a Mendoza, el algoritmo calcula correctamente el camino por San Luis con 1060 km en lugar de la alternativa por Rosario y Cordoba de 1300 km. Con esto queda cubierta la logica de distancia mas corta y le doy paso a mi compañero para explicar la ruta de mayor flujo."